pontIA

MÁSTER INGENIERÍA DE DATOS

Consultas 5

Funciones y Procedimientos Almacenados - Recursos

Daniel Trabas

pontIA

Índice

Objetivo 3

Anexos 12

PontIA

Objetivo

En esta unidad se detallan los principios fundamentales del control de transacciones lógicas (garantías ACID) y el desarrollo de código procedural (funciones, procedimientos y llamadas) en PostgreSQL, utilizando como base de datos demodb y su esquema bookings.

Control de Transacciones con BEGIN, COMMIT, y ROLLBACK

En los sistemas relacionales modernos, una transacción es una unidad lógica de trabajo que agrupa múltiples sentencias SQL de lectura o escritura. Según la documentación oficial de PostgreSQL, las transacciones se fundamentan en las garantías ACID, dónde se indica que la Atomicidad (Atomicity), garantiza que una transacción sea una unidad indivisible. O se confirman todos los cambios o no se aplica ninguno.

Por otro lado en dicha documentación, podemos leer:

> 'Las transacciones son un concepto fundamental de todos los sistemas de bases de datos. El punto esencial de una transacción es que agrupa múltiples pasos en una operación de todo o nada. Los estados intermedios entre los pasos no son visibles para otras transacciones concurrentes, y si ocurre alguna falla que impida que la transacción se complete, entonces ninguno de los pasos afectará la base de datos en absoluto.' (PostgreSQL Documentation, p. 18 / Pasaje 550)

PontIA

Instrucciones Fundamentales de Control

- **BEGIN (o START TRANSACTION):** inicia el bloque de transacción, aislando temporalmente las operaciones del resto de la base de datos.
- **COMMIT (o END):** confirma y aplica de manera irreversible todos los cambios en el almacenamiento físico, garantizando la Durabilidad.
- **ROLLBACK (o ABORT):** aborta el bloque transaccional, descartando y revirtiendo cualquier modificación realizada desde el BEGIN.
- **SAVEPOINT [nombre]:** sirve para marcar un punto de guardado de cambios de transacción. Así mismo esta cláusula cuenta con:
  - ROLLBACK TO SAVEPOINT [nombre] confirma la transacción del punto de guardado.
  - RELEASE SAVEPOINT [nombre] para anular la transacción del punto de guardado.

Cómo ejecutar los scripts para ver el comportamiento transaccional

Cuando queramos comprobar la forma en la que las cláusulas de validación o cancelación de transacciones funcionan, debemos tener en cuenta que no podemos ejecutar las mismas como un bloque. Es por ello que para los ejemplos y ver cómo funcionan, ejecutaremos primero BEGIN luego el resto de las consultas y por último el COMMIT o el ROLLBACK.,

Ejemplos prácticos sobre demodb

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

Insertamos con COMMIT seguro

```
-- 1. Creamos un registro en la tabla bookings.bookings con book_ref = 'XJK300'
BEGIN;
  INSERT INTO bookings.bookings (book_ref, book_date, total_amount)
  VALUES ('XJK300', '2024-06-01', 500.00);
-- Verificamos que el registro se haya insertado correctamente
/*
  Si abrimos otra sesión de base de datos y ejecutamos la siguiente consulta,
  no veremos el registro insertado hasta que se haga COMMIT.
*/
  SELECT bk.*
  FROM    bookings.bookings bk
  WHERE bk.book_ref = 'XJK300';
-- Validamos la transacción y confirmamos los cambios realizados en la base de
datos
COMMIT;
```

Transacción Multi-Tabla consistente con COMMIT

```
-- 2. Insertamos un registro en la tabla bookings.bookings y otro en la tabla
bookings.tickets, y luego hacemos un COMMIT para confirmar los cambios
realizados en ambas tablas.
BEGIN;
  -- Insertar registro en la tabla cabecera de reservas
  INSERT INTO bookings.bookings (book_ref, book_date, total_amount)
  VALUES ('TX999A', CURRENT_TIMESTAMP, 15000.00);
  -- Insertar el billete del pasajero asociado a la clave primaria generada
  INSERT INTO bookings.tickets (ticket_no, book_ref, passenger_id,
passenger_name, outbound)
  VALUES ('9999999999999', 'TX999A', 'ES 4383871116867', 'Alberto Gomez',
'True');
COMMIT;
-- Validamos los dos registros fuera de la transacción para confirmar que se
han insertado correctamente
SELECT * FROM bookings.bookings WHERE book_ref = 'TX999A';
SELECT * FROM bookings.tickets WHERE book_ref = 'TX999A';
-- Ahora ejecutamos un UPDATE sobre bookings.tickets
BEGIN;
  UPDATE bookings.tickets
  SET passenger_name = 'Alberto Gomez Martinez'
  WHERE book_ref = 'TX999A';
  -- Validamos el cambio realizado en la tabla bookings.tickets
  SELECT * FROM bookings.tickets WHERE book_ref = 'TX999A';
  -- Confirmamos los cambios realizados en la tabla bookings.tickets
COMMIT;
```

Prevención de cambios masivos con tabla temporal y SAVEPOINT

```
-- 3. Insertamos un registro en la tabla bookings.bookings y otro en la tabla
bookings.tickets, y luego hacemos un ROLLBACK para deshacer los cambios
realizados en ambas tablas.
/*
-- Solo para DOCENTE:
DROP TABLE IF EXISTS bookings.tickets_it_temp;
*/
BEGIN;
  CREATE TABLE bookings.tickets_it_temp AS
  SELECT DISTINCT bk.book_date::date AS book_date,
          fl.scheduled_departure,
          fl.flight_id,
          fl.route_no,
          ad.airplane_code,
          rt.arrival_airport,
          ap.airport_name AS arrival_airport_name,
          tk.*,
          sg.fare_conditions,
          bk.total_amount
  FROM    bookings.tickets tk
    INNER JOIN bookings.bookings bk ON tk.book_ref = bk.book_ref
    INNER JOIN bookings.segments sg ON tk.ticket_no = sg.ticket_no
    INNER JOIN bookings.flights fl ON sg.flight_id = fl.flight_id
    INNER JOIN bookings.routes rt ON fl.route_no = rt.route_no
    INNER JOIN bookings.airplanes_data ad ON rt.airplane_code =
ad.airplane_code
    INNER JOIN bookings.airports ap ON rt.arrival_airport = ap.airport_code
  WHERE   tk.passenger_id LIKE 'IT%'
    AND   tk.outbound = 'False'
    AND   bk.book_date::date = '2026-01-01';
  -- Validamos el tipo de asiento por avión para todos los vuelos registrados
en la tabla bookings.tickets_temp
  SELECT airplane_code,
          SUM(CASE WHEN fare_conditions = 'Business' THEN 1 ELSE 0 END) AS
business_records,
          SUM(CASE WHEN fare_conditions = 'Comfort' THEN 1 ELSE 0 END) AS
comfort_records,
          SUM(CASE WHEN fare_conditions = 'Economy' THEN 1 ELSE 0 END) AS
economy_records
  FROM    bookings.tickets_it_temp
  WHERE   airplane_code IN ('77W', '351')
  GROUP BY airplane_code
  ORDER BY airplane_code;

  -- Vamos a cambiar el tipo de avión de la tabla bookings.airplanes_data para
el avión con código '77W' a '351' y luego haremos un ROLLBACK para deshacer el
cambio realizado.
  UPDATE bookings.tickets_it_temp
  SET airplane_code = '77W'
  WHERE airplane_code = '351';
  -- Validamos cambio realizado en la tabla bookings.tickets_it_temp
  SELECT airplane_code,
```

```
          SUM(CASE WHEN fare_conditions = 'Business' THEN 1 ELSE 0 END) AS
business_records,
          SUM(CASE WHEN fare_conditions = 'Comfort' THEN 1 ELSE 0 END) AS
comfort_records,
          SUM(CASE WHEN fare_conditions = 'Economy' THEN 1 ELSE 0 END) AS
economy_records
  FROM    bookings.tickets_it_temp
  WHERE   airplane_code IN ('77W', '351')
  GROUP BY airplane_code
  ORDER BY airplane_code;
  -- Ahora vamos a crear un punto de guardado con nombre "it_temp_savepoint"
para poder deshacer los cambios realizados en la tabla bookings.tickets_temp
  SAVEPOINT it_temp_savepoint;
    -- Vemos la situación de los registros por tipo de asiento en la tabla
bookings.tickets_temp antes de realizar el UPDATE
    SELECT COUNT(*) AS total_records,
           SUM(CASE WHEN fare_conditions = 'Business' THEN 1 ELSE 0 END) AS
business_records,
           SUM(CASE WHEN fare_conditions = 'Comfort' THEN 1 ELSE 0 END) AS
comfort_records,
           SUM(CASE WHEN fare_conditions = 'Economy' THEN 1 ELSE 0 END) AS
economy_records
    FROM    bookings.tickets_it_temp;
    -- Realizamos un UPDATE sobre la tabla bookings.tickets_temp para cambiar
el valor de la columna fare_conditions de 'Confort' a 'Business'
    UPDATE bookings.tickets_it_temp
    SET     fare_conditions = 'Business'
    WHERE   fare_conditions = 'Comfort';
    -- Validamos el cambio realizado en la tabla bookings.tickets_temp
    SELECT COUNT(*) AS total_records,
           SUM(CASE WHEN fare_conditions = 'Business' THEN 1 ELSE 0 END) AS
business_records,
           SUM(CASE WHEN fare_conditions = 'Comfort' THEN 1 ELSE 0 END) AS
comfort_records,
           SUM(CASE WHEN fare_conditions = 'Economy' THEN 1 ELSE 0 END) AS
economy_records
    FROM    bookings.tickets_it_temp;
  -- Deshacemos los cambios realizados en la tabla bookings.tickets_temp hasta
el punto de guardado "it_temp_savepoint"
  ROLLBACK TO SAVEPOINT it_temp_savepoint;

/*
  Sin en mitad del bloque de transacción, queremos validar los cambios hasta el
punto de guardado "it_temp_savepoint"
  pero sin terminar de confirmar el resto de la transacción podemos usasr:
  RELEASE SAVEPOINT it_temp_savepoint;
*/
  -- Validamos que los cambios realizados en la tabla bookings.tickets_temp se
hayan deshecho correctamente
  SELECT airplane_code,
         SUM(CASE WHEN fare_conditions = 'Business' THEN 1 ELSE 0 END) AS
business_records,
```

```
          SUM(CASE WHEN fare_conditions = 'Comfort' THEN 1 ELSE 0 END) AS
comfort_records,
          SUM(CASE WHEN fare_conditions = 'Economy' THEN 1 ELSE 0 END) AS
economy_records
  FROM    bookings.tickets_it_temp
  WHERE   airplane_code IN ('77W', '351')
  GROUP BY airplane_code
  ORDER BY airplane_code;
-- Podemos confirmar el resto de los cambios realizados en la tabla
bookings.tickets_temp
COMMIT;
```

Definición y Uso de Funciones (CREATE FUNCTION)

En PostgreSQL, una función es un bloque de instrucciones reutilizable que acepta parámetros de entrada y devuelve obligatoriamente un valor de retorno o un conjunto de filas estructuradas. Las funciones se utilizan principalmente para encapsular consultas lógicas de lectura frecuente, operaciones matemáticas complejas o manipulación de datos estructurados.

Se ejecutan siempre dentro de una transacción ya abierta; por lo tanto, no pueden realizar confirmaciones (COMMIT) o cancelaciones (ROLLBACK) independientes dentro de su cuerpo.

Tipos de Funciones según el Lenguaje de Programación:

- **Funciones SQL:** diseñadas para ejecutar sentencias declarativas directas. El compilador de PostgreSQL las optimiza y las procesa 'inline', integrando la consulta en el plan de ejecución principal.
- **Funciones PL/pgSQL:** utilizan el lenguaje procedural propio del motor. Permiten control de flujo (bucles, condicionales), manipulación dinámica y gestión de excepciones avanzada.

Ejemplos prácticos sobre demodb

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

Obtener importe formateado de una reserva (Lenguaje SQL)

```
-- 1. Creamos la función bookings.get_formatted_amount que recibe como
parámetro el book_ref de la reserva y devuelve el importe total de la reserva
en formato monetario.
CREATE OR REPLACE FUNCTION bookings.get_formatted_amount (p_book_ref
varchar(6))
RETURNS text
AS $$
  SELECT TO_CHAR(bk.total_amount, 'LFM999,999,990.00')
  FROM    bookings.bookings bk
  WHERE   bk.book_ref = p_book_ref;
$$ LANGUAGE SQL;

-- Ejecutamos la función creada para obtener el importe total de la reserva con
book_ref = '13NH0K' en formato monetario
SELECT bookings.get_formatted_amount('13NH0K') AS formatted_amount;
-- Podemos ver la diferencia entre el resultado de la función y el resultado de
la consulta directa a la tabla bookings.bookings
SELECT * FROM bookings.bookings bk WHERE bk.book_ref = '13NH0K';
```

Historial de vuelos por pasajero (Lenguaje PL/pgSQL y RETURNS TABLE)

```
-- 2. Creamos la función bookings.get_passenger_flight_history que recibe como
parámetro el passenger_id del pasajero
CREATE OR REPLACE FUNCTION bookings.get_passenger_flight_history
(p_passenger_id varchar(100))
RETURNS TABLE (
  ticket_no text,
  passenger_name text,
  flight_id integer,
  route_no text,
  status text,
  segment_price numeric(10,2)
)
AS $$
BEGIN RETURN QUERY
  SELECT tk.ticket_no,
         tk.passenger_name,
         fl.flight_id,
         fl.route_no,
         fl.status,
         sg.price
  FROM bookings.tickets tk
    INNER JOIN bookings.segments sg ON tk.ticket_no = sg.ticket_no
    INNER JOIN bookings.flights fl ON sg.flight_id = fl.flight_id
  WHERE tk.passenger_id = p_passenger_id;
END;
$$ LANGUAGE plpgsql;

-- Ejecutamos la función creada para obtener el historial de vuelos del
pasajero que indiquemos
SELECT * FROM bookings.get_passenger_flight_history('US 2779671121032');
```

Diseño de Procedimientos Almacenados (CREATE PROCEDURE)

A partir de versiones recientes[1], PostgreSQL incorporó la instrucción CREATE PROCEDURE. La diferencia fundamental y más crítica respecto a las funciones es que los procedimientos pueden iniciar, confirmar o revertir transacciones internas de forma autónoma (mediante comandos COMMIT y ROLLBACK explícitos dentro de su bloque de código).

Esto los hace indispensables para el procesamiento masivo de datos por lotes (ETL) o el mantenimiento programado, ya que previenen la sobrecarga de la memoria RAM o bloqueos en producción.

[1] Los procedimientos almacenados (PROCEDURE) se introdujeron oficialmente en PostgreSQL a partir de la versión 11, lanzada en octubre de 2018.Antes de esta versión, la única opción disponible eran las funciones (FUNCTION), las cuales no permitían el control de transacciones (COMMIT/ROLLBACK) en su interior

Ejemplos prácticos sobre demodb

En los siguientes ejemplos únicamente realizaremos la creación de los procedimientos almacenados, dejando para el siguiente bloque el cómo ejecutarlos.

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

Ajuste de tarifas por vuelo con COMMIT autónomo

```
-- 1. Creamos el procedimiento bookings.update_flight_segment_prices que recibe
como parámetros el flight_id del vuelo y un porcentaje de ajuste para
actualizar los precios de los segmentos de ese vuelo.
CREATE OR REPLACE PROCEDURE bookings.update_flight_segment_prices (
  p_flight_id integer,
  p_adjustment_percentage numeric,
  p_fare_conditions text DEFAULT NULL
) AS $$
BEGIN
  UPDATE bookings.segments sg
  SET price = sg.price * (1 + p_adjustment_percentage / 100.0)
  WHERE sg.flight_id = p_flight_id
    AND (p_fare_conditions IS NULL OR sg.fare_conditions = p_fare_conditions);
  COMMIT; --Confirmación física e independiente de la transacción
END;
$$ LANGUAGE plpgsql;
```

Archivo de vuelos cancelados con auditoría de pasos y retorno de tipo cursor

PontIA

```
-- 2. Creamos el procedimiento bookings.archive_cancelled_flights que recibe
como parámetros el route_no de la ruta y un cursor lógico para devolver los
registros archivados.
CREATE OR REPLACE PROCEDURE bookings.archive_cancelled_flights (
  p_route_no text,
  p_result_cursor INOUT refcursor
) AS $$
DECLARE v_count integer;
BEGIN
  RAISE NOTICE 'Iniciando proceso de archivo de vuelos cancelados para la ruta:
%', p_route_no;
  -- Paso 1: Crear una tabla temporal para la auditoría si no existe
  CREATE TEMPORARY TABLE IF NOT EXISTS temp_cancelled_archive (
    flight_id integer,
    route_no text,
    status text,
    archived_at timestamp DEFAULT CURRENT_TIMESTAMP
  ) ON COMMIT PRESERVE ROWS;
  RAISE NOTICE 'Tabla temporal verificada/creada con éxito.';
  -- Paso 2: Contar registros a procesar
  SELECT COUNT(*)
  INTO v_count
  FROM bookings.flights fl
  WHERE fl.route_no = p_route_no
    AND fl.status = 'Cancelled';
  RAISE NOTICE 'Se han encontrado % vuelos cancelados para procesar.', v_count;
  IF v_count > 0 THEN
  -- Paso 3: Insertar en la tabla histórica
    INSERT INTO temp_cancelled_archive (flight_id, route_no, status)
    SELECT fl.flight_id,
           fl.route_no,
           fl.status
    FROM    bookings.flights fl
    WHERE   fl.route_no = p_route_no
      AND   fl.status = 'Cancelled';
    RAISE NOTICE 'Registros copiados exitosamente a la tabla de archivo.';
  ELSE
    RAISE NOTICE 'No se requirieron operaciones de inserción.';
  END IF;
  -- Paso 4: Abrir el cursor de salida para que el cliente SQL consuma la
información
  OPEN p_result_cursor FOR
      SELECT *
      FROM    temp_cancelled_archive
      WHERE   route_no = p_route_no;
    RAISE NOTICE 'Cursor de salida abierto de manera exitosa.';
END;
$$ LANGUAGE plpgsql;
```

Comparativa técnica entre Funciones y Procedimientos Almacenados

A continuación se presenta un cuadro comparativo detallado que resume las diferencias de diseño y escenarios de uso recomendados en producción para cada uno de estos dos objetos procedurales:

Característica	CREATE FUNCTION	CREATE PROCEDURE
Control Transaccional	No permitido. No puede invocar COMMIT o ROLLBACK en su bloque.	Permitido. Puede gestionar y confirmar transacciones internas libremente.
Retorno de Valores	Obligatorio. Mediante cláusula RETURNS (un tipo escalar, compuesto o RETURNS TABLE).	No utiliza cláusula de retorno tradicional. Devuelve datos con parámetros INOUT/OUT.
Ejecución Lógica	Invocada directamente en sentencias SQL lógicas (como SELECT, WHERE, etc.).	Invocada de manera independiente mediante el comando CALL.
Uso Óptimo	Cálculos matemáticos, filtros lógicos, formateo de texto, vistas dinámicas o consultas repetitivas.	Procesos ETL por lotes, mantenimientos periódicos, manipulación masiva de registros.
Estado de Datos	Diseñada fundamentalmente para ser de sólo lectura o con cambios transaccionales ligados.	Estructurada para procesar de forma secuencial grandes bloques con transacciones independientes.

CALL

La sentencia CALL es el comando estándar introducido en SQL para invocar un procedimiento almacenado registrado en el catálogo del sistema. No puede ser llamada como parte de una sentencia SELECT ordinaria, sino que debe ejecutarse como una instrucción independiente.

Ejemplos prácticos sobre demodb

A continuación se mostrará la ejecución de los procedimientos almacenados creados anteriormente.

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

```
-- 1. Ejecutaremos el procedimiento bookings.update_flight_segment_prices
creado anteriormente
CALL bookings.update_flight_segment_prices(12345, -10.00, 'Economy');

-- Validamos el cambio realizado en la tabla bookings.segments para el vuelo
con flight_id = 1001
SELECT *
FROM    bookings.segments
WHERE   flight_id = 12345
  AND   fare_conditions = 'Economy';

-- 2. Ejecutaremos el procedimiento bookings.archive_cancelled_flights creado
anteriormente
BEGIN;
  CALL bookings.archive_cancelled_flights('PG0805', 'mi_cursor');
  FETCH ALL FROM mi_cursor;
COMMIT;
```

Anexos

Enlaces de apoyo para los contenidos

Para profundizar más en todos los contenidos de esta unidad, se facilitan las siguientes referencias directas.

- Documentación Oficial de PostgreSQL (Transactions): https://www.postgresql.org/docs/current/tutorial-transactions.html
- Guía de Referencia de CREATE FUNCTION: https://www.postgresql.org/docs/current/sql-createfunction.html
- Guía de Referencia de CREATE PROCEDURE: https://www.postgresql.org/docs/current/sql-createprocedure.html
- Comando CALL en PostgreSQL: https://www.postgresql.org/docs/current/sql-call.html

¿Qué sucede cuando se ejecuta un procedimiento almacenado?

Encapsular la ejecución de consultas pesadas en procedimientos almacenados para una aplicación cuyo backend utilice una base de datos PostgreSQL, permitirá un ahorro de RAM y la mejora de la ejecución de las consultas.

Cuando se ejecuta un procedimiento almacenado por primera vez en una sesión, PostgreSQL analiza el código SQL interno y genera lo que se conoce como Planes Preparados (Prepared Plans) para cada consulta (SELECT, INSERT, UPDATE, etc.) que está dentro del procedimiento.

Al hacer esto, el motor de la base de datos se ahorra tres pasos costosos en las siguientes llamadas:

- **Parsing:** analiza la sintaxis de la consulta
- **Analyzing:** valida las tablas y columnas existentes
- **Planning:** calcula la ruta más rápida (índices a usar, tipos de joins, etc.) para traer los datos

Plan Genérico vs. Plan Específico

PostgreSQL tiene un comportamiento dinámico muy inteligente:

- **Primeras 5 ejecuciones:** PostgreSQL genera un Plan Específico optimizado para los argumentos exactos que se le ha pasado en la llamada.
- **A partir de la 6ª ejecución:** el motor compara el costo de seguir haciendo planes específicos contra el costo de usar un Plan Genérico (un plan que sirve para cualquier parámetro). Si el plan genérico es igual de eficiente o la diferencia es mínima, PostgreSQL cachea el plan genérico permanentemente para esa sesión.

Estos planes únicamente se mantienen en la caché para la sesión que los ha ejecutado (otros motores de SQL como MSSQL[2] lo hacen de manera global). Es por ello que siempre que se quiera mejorar el rendimiento de una aplicación con PostgreSQL en su backend, se recomienda el uso de un pool de conexiones (como PgBouncer o el de tu framework backend). De esta forma, estas sesiones se mantienen vivas por mucho tiempo, maximizando el uso de esta caché de planes.

[2] Microsoft SQL Server

pontIA